Name: Kamal Sharma

UID: 24BAI70380

Course: BE-CSE (AI&ML)

Subject: Database Management System

Experiment: Design and Implementation of Materialized Views

1. Aim of the Session

To design and implement a materialized view and to compare and analyze execution time and performance differences between simple views, complex views, and materialized views, thereby understanding their impact on query optimization and system performance.

2. Software Requirements

- **Database**:
  - Oracle live SQL
  - PostgreSQL Database (PgAdmin)

3. Objective of the Session

By the end of the session, the following objectives were achieved:

- To create simple views, complex views, and materialized views.
- To evaluate performance by comparing query execution time for each.
- To highlight the advantages of materialized views in enterprise-level applications.
- To understand how materialized views improve query optimization and system performance.

4. Practical / Experiment Steps

The experiment was carried out through the following activities:

- Create tables with multiple records.
- Create a simple view based on a single table.
- Create a complex view involving joins, filters, or aggregations.
- Create a materialized view storing precomputed query results.
- Analyze and compare their execution time and performance.
- Print results or apply business logic for each record.
- Execute PL/SQL blocks and verify correctness of row-by-row processing.
- Save PL/SQL scripts and outputs for reporting.

5. Procedure of the Practical

Execution was performed in the following order:

1. **Environment Setup:** Logged into Oracle LIVE SQL Developer to prepare the workspace.
2. **Table Creation:** Created EMPLOYEE and DEPARTMENT tables.

```sql
CREATE TABLE DEPARTMENT (
    DEPT_ID INT PRIMARY KEY,
    DEPT_NAME VARCHAR(50)
);

CREATE TABLE EMPLOYEE (
    EMP_ID INT PRIMARY KEY,
    EMP_NAME VARCHAR(50),
    DEPT_ID INT,
    SALARY NUMERIC,
    CONSTRAINT fk_dept FOREIGN KEY (DEPT_ID)
        REFERENCES DEPARTMENT(DEPT_ID)
);
```

3. **Data Insertion:** inserted of data in both tables.

```sql
INSERT INTO DEPARTMENT VALUES (1, 'HR');

INSERT INTO DEPARTMENT VALUES (2, 'Finance');

INSERT INTO DEPARTMENT VALUES (3, 'IT');

INSERT INTO EMPLOYEE VALUES (101, 'Rahul', 1, 25001);

INSERT INTO EMPLOYEE VALUES (102, 'Priya', 2, 28000);

INSERT INTO EMPLOYEE VALUES (103, 'Aman', 3, 45000);

INSERT INTO EMPLOYEE VALUES (104, 'Sneha', 2, 32003);

INSERT INTO EMPLOYEE VALUES (105, 'Karan', 1, 15000);

INSERT INTO EMPLOYEE VALUES (106, 'Meena', 3, 17501);
```

4. Implementation of **simple view** on a single table.

```sql
-- Task-1: Simple View

CREATE VIEW simple_emp_view AS
SELECT EMP_ID, EMP_NAME, SALARY
FROM EMPLOYEE;

SELECT * FROM simple_emp_view;
```

5. Implementation of **complex view** with joins and aggregations.

```sql
-- Task-2: Complex View

CREATE VIEW complex_emp_view AS
SELECT d.DEPT_NAME, AVG(e.SALARY) AS avg_salary, COUNT(e.EMP_ID) AS total_employees
FROM EMPLOYEE e
JOIN DEPARTMENT d ON e.DEPT_ID = d.DEPT_ID
GROUP BY d.DEPT_NAME;

SELECT * FROM complex_emp_view;
```

6. Implementation of **materialized view** to store precomputed results.

```sql
-- Task-3: Materialized View

CREATE MATERIALIZED VIEW mv_emp_salary_pg AS
SELECT d.DEPT_NAME, AVG(e.SALARY) AS avg_salary, COUNT(e.EMP_ID) AS total_employees
FROM EMPLOYEE e
JOIN DEPARTMENT d ON e.DEPT_ID = d.DEPT_ID
GROUP BY d.DEPT_NAME;

REFRESH MATERIALIZED VIEW mv_emp_salary_pg;

SELECT * FROM mv_emp_salary_pg;
```

7. **Implementation of Performance Analysis**

```sql
-- Task-4: Performance Analysis

EXPLAIN ANALYZE SELECT * FROM simple_emp_view;
EXPLAIN ANALYZE SELECT * FROM complex_emp_view;
EXPLAIN ANALYZE SELECT * FROM mv_emp_salary_pg;
```

8. **Output Verification: Executed the block and verified the output for correctness.**
9. **Documentation: Saved the SQL script and captured outputs for reporting and future reference.**

6. I/O Analysis (Input / Output Analysis)

Input SQL Queries

```sql
 CREATE TABLE DEPARTMENT (

    DEPT_ID INT PRIMARY KEY,

    DEPT_NAME VARCHAR(50)

);



CREATE TABLE EMPLOYEE (

    EMP_ID INT PRIMARY KEY,

    EMP_NAME VARCHAR(50),

    DEPT_ID INT,
```

```
    SALARY NUMERIC,
    CONSTRAINT fk_dept FOREIGN KEY (DEPT_ID)
        REFERENCES DEPARTMENT(DEPT_ID)
);

INSERT INTO DEPARTMENT VALUES (1, 'HR');

INSERT INTO DEPARTMENT VALUES (2, 'Finance');

INSERT INTO DEPARTMENT VALUES (3, 'IT');

INSERT INTO EMPLOYEE VALUES (101, 'Rahul', 1, 25001);

INSERT INTO EMPLOYEE VALUES (102, 'Priya', 2, 28000);

INSERT INTO EMPLOYEE VALUES (103, 'Aman', 3, 45000);

INSERT INTO EMPLOYEE VALUES (104, 'Sneha', 2, 32003);

INSERT INTO EMPLOYEE VALUES (105, 'Karan', 1, 15000);

INSERT INTO EMPLOYEE VALUES (106, 'Meena', 3, 17501);


-- Task-1: Simple View
```

```
CREATE VIEW simple_emp_view AS

SELECT EMP_ID, EMP_NAME, SALARY

FROM EMPLOYEE;



SELECT * FROM simple_emp_view;



-- Task-2: Complex View



CREATE VIEW complex_emp_view AS

SELECT d.DEPT_NAME, AVG(e.SALARY) AS avg_salary, COUNT(e.EMP_ID) AS
total_employees

FROM EMPLOYEE e

JOIN DEPARTMENT d ON e.DEPT_ID = d.DEPT_ID

GROUP BY d.DEPT_NAME;



SELECT * FROM complex_emp_view;





-- Task-3: Materialized View



CREATE MATERIALIZED VIEW mv_emp_salary_pg AS

SELECT d.DEPT_NAME, AVG(e.SALARY) AS avg_salary, COUNT(e.EMP_ID) AS
total_employees

FROM EMPLOYEE e

JOIN DEPARTMENT d ON e.DEPT_ID = d.DEPT_ID

GROUP BY d.DEPT_NAME;
```

```
REFRESH MATERIALIZED VIEW mv_emp_salary_pg;


SELECT * FROM mv_emp_salary_pg;


-- Task-4: Performance Analysis


EXPLAIN ANALYZE SELECT * FROM simple_emp_view;

EXPLAIN ANALYZE SELECT * FROM complex_emp_view;

EXPLAIN ANALYZE SELECT * FROM mv_emp_salary_pg;
```

Output

Task 1:

	emp_id integer	emp_name character varying (50)	salary numeric
1	101	Rahul	25001
2	102	Priya	28000
3	103	Aman	45000
4	104	Sneha	32003
5	105	Karan	15000
6	106	Meena	17501

Task 2:

	dept_name character varying (50)	avg_salary numeric	total_employees bigint
1	Finance	30001.500000000000	2
2	IT	31250.500000000000	2
3	HR	20000.500000000000	2

Task 3:

	dept_name character varying (50)	avg_salary numeric	total_employees bigint
1	Finance	30001.500000000000	2
2	IT	31250.500000000000	2
3	HR	20000.500000000000	2

Task 4:

Showing rows: 1 to 4

	QUERY PLAN text
1	Seq Scan on employee (cost=0.00..14.30 rows=430 width=154) (actual time=0.015..0.017 rows=6.00 loops=...
2	Buffers: shared hit=1
3	Planning Time: 0.084 ms
4	Execution Time: 0.050 ms

QUERY PLAN text
HashAggregate (cost=40.81..43.31 rows=200 width=158) (actual time=0.051..0.053 rows=3.00 loops=1)
Group Key: d.dept_name
Batches: 1 Memory Usage: 32kB
Buffers: shared hit=2
-> Hash Join (cost=22.15..37.59 rows=430 width=154) (actual time=0.035..0.039 rows=6.00 loops=1)
Hash Cond: (e.dept_id = d.dept_id)
Buffers: shared hit=2
-> Seq Scan on employee e (cost=0.00..14.30 rows=430 width=40) (actual time=0.016..0.017 rows=6.00 loops=1)
Buffers: shared hit=1
-> Hash (cost=15.40..15.40 rows=540 width=122) (actual time=0.012..0.012 rows=3.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 9kB
Buffers: shared hit=1
-> Seq Scan on department d (cost=0.00..15.40 rows=540 width=122) (actual time=0.008..0.008 rows=3.00 loops...
Buffers: shared hit=1
Planning Time: 0.180 ms
Execution Time: 0.098 ms

	QUERY PLAN text
1	Seq Scan on mv_emp_salary_pg (cost=0.00..14.30 rows=430 width=158) (actual time=0.020..0.021 rows=3.00 loops=1)
2	Buffers: shared hit=1
3	Planning Time: 0.073 ms
4	Execution Time: 0.037 ms

7. Learning Outcome

- Understood the concept and working of materialized views in a database system.
- Differentiated between simple views, complex views, and materialized views.
- Learned how to create and refresh materialized views in PostgreSQL.
- Measured and compared query execution time for different types of views.
- Analyzed performance benefits of materialized views in data-intensive applications.
- Applied materialized view concepts in real-world company scenarios such as SanDisk, JTG, and PayPal.